

E2E AWS-Based Text Classification

End-to-End AWS-Based Text Classification System

Project Overview:

The **End-to-End AWS-Based Text Classification System** project aims to build a text classification solution using AWS services, from data preprocessing and model training to deployment and real-time prediction. This solution will leverage machine learning techniques to classify text data for applications like sentiment analysis, topic classification, or spam detection. The project will implement best practices in MLOps, cloud deployment, and automation for a robust, scalable solution.

Milestone 1: Data Collection, Exploration, and Preprocessing

Objectives:

- Set up AWS environment and prepare text data for model development.

Tasks:

1. AWS Setup:

- Set up AWS services:
 - **Amazon S3:** Store datasets for text classification.
 - **Amazon EC2:** Provision compute resources for data processing and model training.
- Configure IAM roles for secure access to resources.

2. Data Collection:

- Choose an appropriate dataset for the task (e.g., **Sentiment Analysis Dataset, AG News Dataset**).
- Upload the dataset to **Amazon S3** for storage and access.

3. Data Exploration & Preprocessing:

- Conduct exploratory data analysis (EDA) to understand the dataset's structure, class distribution, and any data imbalances.
- Clean and preprocess the text data (e.g., remove stopwords, tokenization, and stemming/lemmatization).
- Split the data into training, validation, and test sets.

Deliverables:

- **AWS Environment Setup:** A fully configured AWS environment with S3 and EC2 set up.
- **Preprocessed Text Dataset:** A cleaned and tokenized dataset ready for model training.
- **EDA Report:** A summary of insights from data exploration.

Milestone 2: Text Classification Model Development

Objectives:

- Develop and train a machine learning model for text classification.

Tasks:

1. Model Selection:

- Choose an appropriate model for text classification (e.g., **Logistic Regression**, **Naive Bayes**, **Random Forest**, or more advanced methods like **BERT** for NLP tasks).

2. Model Development using AWS SageMaker:

- Use **AWS SageMaker** to create and train the model.
- For traditional models, use **Scikit-learn** with **TF-IDF** for feature extraction.
- If using transformer models, integrate **Hugging Face** with SageMaker to leverage pre-trained models like **BERT** or **DistilBERT**.

3. Model Training & Evaluation:

- Train the model using the training set and evaluate it on the validation set using classification metrics such as accuracy, precision, recall, F1-score, and confusion matrix.

Deliverables:

- **Trained Model:** A trained text classification model.
- **Evaluation Report:** A detailed report comparing model performance based on evaluation metrics.
- **Model Code:** Python scripts for data preprocessing, feature extraction, model training, and evaluation.

Milestone 3: Model Deployment and Real-Time Prediction Setup

Objectives:

- Deploy the trained model to AWS for real-time text classification.

Tasks:

1. Deploy Model with AWS SageMaker:

- Deploy the trained model to **AWS SageMaker Endpoints** for real-time inference.
- Set up a RESTful API to receive input and return predictions using **API Gateway** and **AWS Lambda**.

2. Integrate Real-Time Predictions:

- Use **AWS Lambda** to handle incoming requests and invoke the model endpoint to make predictions.

- Implement real-time text classification, such as sentiment analysis or topic detection.

3. Testing and Optimization:

- Test the deployed model with different types of input (e.g., text data from various domains).
- Optimize latency and cost for real-time inference.

Deliverables:

- **Deployed Model Endpoint:** A live model endpoint on SageMaker for real-time predictions.
- **Lambda Function:** A Lambda function integrated with the model endpoint to handle text classification requests.
- **API Gateway Setup:** A secure API for real-time interaction with the deployed model.

Milestone 4: MLOps Integration and Monitoring

Objectives:

- Implement MLOps practices for model monitoring, versioning, and retraining.

Tasks:

1. MLOps Setup with MLflow:

- Set up **MLflow** to track experiments, store models, and manage model versions.
- Implement version control for the model to track changes and updates over time.

2. Model Monitoring:

- Implement continuous monitoring of model performance (e.g., accuracy, latency, error rates) in real-time production.
- Set up alerts using **Amazon CloudWatch** for tracking system performance and failures.

3. Automated Retraining Pipeline:

- Set up an automated pipeline using **AWS Step Functions** to retrain the model periodically with new data or as model performance drifts over time.
- Implement automatic deployment of new model versions using **AWS CodePipeline**.

Deliverables:

- **MLOps Pipeline:** A working MLOps pipeline using **MLflow** for experiment tracking and versioning.
- **Model Monitoring Setup:** Continuous monitoring and alerting for the model in production.
- **Retraining Pipeline:** An automated system to retrain and redeploy the model.

Milestone 5: Final Documentation and Presentation

Objectives:

- Document the project process, model performance, and deployment, and prepare a final presentation.

Tasks:

1. Final Report:

- Summarize the project, including:
 - AWS Setup:** Details on the setup of AWS services.
 - Model Development:** Description of model selection, training, and evaluation.
 - Deployment:** Explanation of deployment using SageMaker and Lambda.
 - MLOps & Monitoring:** Overview of the MLOps pipeline and model monitoring.
 - Business Impact:** How this solution can be applied to real-world text classification tasks, such as sentiment analysis, spam detection, or topic categorization.

2. Final Presentation:

- Create a presentation highlighting the technical details, challenges, and business impact of the text classification system.
- Include a live demo of the system in action, showcasing real-time text classification.

Deliverables:

- Final Project Report:** A comprehensive report covering all aspects of the project.
- Final Presentation:** A professional presentation with key insights, architecture diagrams, and demo.
- Live Demo:** A functional demo of the deployed system showcasing real-time text classification.

Final Milestone Summary:

Milestone	Key Deliverables
1. Data Collection, Exploration & Preprocessing	AWS environment setup, Cleaned text dataset, EDA report
2. Model Development	Trained model, Evaluation report, Model training and evaluation code
3. Deployment & Real-Time Prediction	Deployed model endpoint, Lambda function, API Gateway setup
4. MLOps Integration & Monitoring	MLOps pipeline, Model monitoring setup, Retraining pipeline
5. Final Documentation & Presentation	Final report, Final presentation, Live demo

Conclusion:

This **End-to-End AWS-Based Text Classification System** project provides a complete workflow for developing, deploying, and maintaining a real-time text classification model. The project uses AWS tools like **SageMaker**, **Lambda**, **S3**, and **API Gateway** to create a scalable, production-ready system. With MLOps integration, the system ensures continuous monitoring and updates, while providing a solid foundation for future enhancements or new use cases.